A Grammar
for the Tiger Programming Language

Group ID: **G46**

Submitted by

Name	**ID**
Lokesh L	2022B4A71618H
R Tarun Kumar	2022B3A71606H
Anuj R	2022B5A70899H
M Mahadevan	2022B3A70641H
Pranav P Jagathpathy	2022B4A70761H

Birla Institute of Technology and Science, Pilani - Hyderabad Campus

CS F363: Compiler Construction

May 25, 2026

1 Context-Free Grammar

Notation. Nonterminals are written in angular brackets (e.g. ⟨*expr*⟩). Terminals (which are tokens produced by the lexer) are written in monospace (e.g. `array`). Tokens that also carry an attribute are written in uppercase (e.g. `INTEGER`). The empty string is denoted by ϵ.

⟨*program*⟩ ::= ⟨*expr*⟩

⟨*expr*⟩ ::= `INTEGER`
| `STRING`
| `nil`
| ⟨*lvalue*⟩
| `-` ⟨*expr*⟩
| ⟨*expr*⟩ ⟨*binary-op*⟩ ⟨*expr*⟩
| ⟨*lvalue*⟩ `:=` ⟨*expr*⟩
| `IDENTIFIER [` ⟨*expr*⟩ `] of` ⟨*expr*⟩
| `if` ⟨*expr*⟩ `then` ⟨*expr*⟩
| `if` ⟨*expr*⟩ `then` ⟨*expr*⟩ `else` ⟨*expr*⟩
| `while` ⟨*expr*⟩ `do` ⟨*expr*⟩
| `break`
| `(` ⟨*expr-seq-opt*⟩ `)`
| `let` ⟨*decl-list*⟩ `in` ⟨*expr-seq-opt*⟩ `end`

⟨*expr-seq*⟩ ::= ⟨*expr*⟩
| ⟨*expr-seq*⟩ `;` ⟨*expr*⟩

⟨*expr-seq-opt*⟩ ::= ⟨*expr-seq*⟩
| ϵ

⟨*lvalue*⟩ ::= `IDENTIFIER`
| `IDENTIFIER [` ⟨*expr*⟩ `]`

⟨*decl-list*⟩ ::= ⟨*decl*⟩
| ⟨*decl-list*⟩ ⟨*decl*⟩

⟨*decl*⟩ ::= `type` ⟨*type-decl-list*⟩
| `var` ⟨*var-decl-list*⟩

⟨*type-decl-list*⟩ ::= ⟨*type-decl*⟩
| ⟨*type-decl-list*⟩ ⟨*type-decl*⟩

⟨*type-decl*⟩ ::= `IDENTIFIER = array of` ⟨*type-id*⟩

⟨*var-decl-list*⟩ ::= ⟨*var-decl*⟩
| ⟨*var-decl-list*⟩ ⟨*var-decl*⟩

⟨*var-decl*⟩ ::= `IDENTIFIER :` ⟨*type-id*⟩
| `IDENTIFIER :` ⟨*type-id*⟩ `:=` ⟨*expr*⟩

⟨*binary-op*⟩ ::= `+`
| `-`
| `*`
| `/`
| `=`
| `<>`
| `<`
| `<=`
| `>`
| `>=`
| `&`
| `|`

⟨*type-id*⟩ ::= `int`
| `string`

2 Lexical Specification

Classification. Each lexeme in Tiger is either a **reserved word**, an **operator**, a **punctuation symbol**, an **identifier** (`IDENTIFIER`), an **integer constant** (`INTEGER`), or a **string constant** (`STRING`).

Resolution Policy. Reserved words, operators and punctuation symbols are matched exactly. Identifiers, integer constants and string constants are matched using regular expressions. The scanner follows the **maximal-munch** rule. Among matches of equal length, priority is given to reserved words.

2.1 Reserved Words

```
array break do else end if in let nil of then type var while
```

2.2 Operators

```
+ - * / = <> < <= > >= & |
```

2.3 Punctuation Symbols

```
:= ( ) [ ] ; :
```

2.4 Identifiers

```
[A-Za-z][A-Za-z0-9_]*
```

2.5 Integer Constants

```
[0-9]+
```

2.6 String Constants

```
"([ -!]|[#-\[]|[\]-~]|\\[nt"\\]|\\\^[@A-Z\[\]\\\^_]|\\[ \t\n]+\\)*"
```